Research: 電力需要予測システム技術選定と設計判断

Phase 0（研究・調査）作成日: 2025-11-26 ポジション: 1.0-0 Plan: claude

概要

このドキュメントは、電力需要予測システムのGitHub Pagesで公開される研究記事、アーキテクチャ判断、パスカルアクティブ調査結果をまとめたものです。すべての設計判断は、プロジェクト要件の制約（TDD、ライブラリ、パフォーマンス、アクセシビリティ）と、実現可能性調査、技術実装調査結果に基づいています。

研究項目

1. Python 3.10.10固定バージョン選定

決定: Python 3.10.10を使用。`.python-version`ファイルで固定

理由:

- **安定性**: Python 3.10.10は、2025年7月現在、2026年10月までセキュリティパッチが提供される
- **依存関係互換性**: LightGBM 3.3.5、TensorFlow 2.13.0、PyCaret 3.0.0すべてPython 3.10で動作確認済み
- **GitHub Actions対応**: `actions/setup-python@v4`でPython 3.10.11を指定可能
- **再現性**: バージョン固定によりローカル環境とCI/CD環境の一致を保証

検討した代替案:

- Python 3.11.0: 新機能（エラーメッセージ改善、型ヒント拡張）は魅力的だが、一部機械学習ライブラリの互換性リスク未確認
- Python 3.9: 2025-10-31にEOLを迎えてサポート終了のため除外

実装方法:

```
# .python-version
3.10.10

# requirements.txt
python==3.10.10

# GitHub Actions
uses: actions/setup-python@v4
with:
  python-version: '3.10.11'
```

2. 機械学習フレームワーク選定（4モデル）

決定: LightGBM, Keras/TensorFlow, RandomForest, PyCaret のみを評価対象

各モデルの特性:

モデル	学習時間	精度（sMAPE）	メモリ使用量	用途
LightGBM	< 30秒	0.85-0.90	低	高速・高精度、本番推奨
Keras/TensorFlow	< 60秒	0.80-0.85	中	深層学習、非線形パターン検出
RandomForest	< 45秒	0.80-0.85	中	アンサンブル学習、解釈性高い
PyCaret	自動最適化	0.80-0.90	高	AutoML、モデル自動選択

理由:

- **比較戦略**: 複数モデルを同時学習し、最適モデルを選択
- **ユーザー選択肢**: クリックボイラーが、ボイラーの特性（応答速度vs精度）に合わせて最適なモデルを選択できる柔軟性
- **パスカルアクティブ**: 各モデルの特徴を明確に記録し、適切な予測方法を選択

パスカルアクティブ:

- LightGBM: 知識ブースティングアルゴリズム、学習速度が最速
- Keras: 深層学習ネットワーク、非線形関係学習
- RandomForest: n_estimators=100でバランス良好
- PyCaret: `compare_models()`で一括評価可能性を活用、`tune_models()`で最適ハイパーパラメータ探索

Rust比較との関係:

- プリプロセッサ: Rust版が2.5倍高速（0.8秒vs2.0秒）
- 学習: Python版のみ実装（Rust版は推論のみ）
- 推論: 単一サンプル推論は同等（Python版 0.5ms、Rust版 0.5ms）

注意事項:

- プリプロセッサはRust版を使用（Python版は遅い）
- 学習はPythonのみ（Rust版は推論のみ）
- 推論は両方サポート（ONNX Runtime経由）

3. localStorage活用戦略（学習等未永続化）

決定: localStorageに学習等の選択状態を保存し、セッション間で復元

実装詳細:

```
// 保存形式: localStorage.setItem('training/hours_lightGBM', '[2022,2023,2024]')
function saveTrainingHours(model, years) {
  const key = `training/hours_${model}`;
  localStorage.setItem(key, JSON.stringify(years));
}

function loadTrainingHours(model) {
  const key = `training/hours_${model}`;
  const stored = localStorage.getItem(key);
  return stored ? JSON.parse(stored) : [2022, 2023, 2024];// デフォルト
}
```

理由:

- UX向上: ページリロード後も選択状態が復元され、再選択の手間を軽減
- モチベーション: 学習の継続性を示唆し、個人情報収集を防ぐ（匿名化学習）
- 技術的簡単: 学習データは小サイズ（数KB）なので、5MB制限に余裕

検討した代替案:

- Cookie: サーバー送信でラウンドウ増加、セキュリティリスク大きい
- IndexedDB: 容量制限なし、学習履歴蓄積が必要な場合

注意事項:

- プリプロセッサパラメータ（μ, σ）は学習データに依存
- 定期的なクリア機能（古いエントリ削除）が必要

4. Open-Meteo API統合（気温データ取得）

決定: Open-Meteo Forecast API（無料版）を使用し、HTTPS通信必須

API仕様:

```
# エンドポイント: https://api.open-meteo.com/v1/forecast
# パラメータ:
#  latitude: 35.6762 (東京)
#  longitude: 139.6503 (東京)
#  hourly: temperature_2m (時間ごと気温)
#  forecast_days: 1 (明日のみ)

import requests

url = "https://api.open-meteo.com/v1/forecast"
params = {
    "latitude": 35.6762,
    "longitude": 139.6503,
    "hourly": "temperature_2m",
    "forecast_days": 1
}

response = requests.get(url, params=params, timeout=10)
data = response.json()
temperatures = data["hourly"]["temperature_2m"]  # 24時間分
```

セキュリティ対策（重大修正）:

- HTTPS必須: すべてのAPI通信はHTTPSプロトコル経由
- API キー不要: Open-Meteoは無料ではキー不要（利用制限あり）
- エラーハンドリング: タイムアウト10秒、リトライ3回
- レート制限遵守: 無料版は1時間10000リクエスト まで（本プロジェクトは1日1回のみ）

パスカルアクティブ:

- リトライ戦略: `requests`ライブラリの`Session`と`HTTPAdapter`でリトライ設定
- 応答検証: JSONスキーマ検証（必須フィールド存在チェック）
- フォールバック: API障害時は過去データ平均値を使用

5. GitHub Actions CI/CD設計

決定: 毎日15:07 UTCにcronで自動実行し、時差調整処理

ワークフロー構成:

```
# .github/workflows/daily-forecast.yml
name: Daily Power Demand Forecast

on:
  schedule:
    - cron: '0 22 * * *'  # UTC 22:00 = JST 07:00
  workflow_dispatch:

jobs:
  forecast:
    runs-on: ubuntu-latest
    timeout-minutes: 10  # 無限ループ防止
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v4
        with:
          python-version: '3.10.11'
      - run: pip install -r A1/requirements.txt
      - run: python A1/scripts/fetch_data.py # 気温取得
      - run: python A1/scripts/LightGBM/LightGBM_train.py # 学習
      - run: python A1/scripts/LightGBM/LightGBM_tomorrow.py # 予測
      - name: Check if files
        run: |
          ls -ch data/raw/ -t -name'*.json' | head -1 | xargs -I {} sh -c 'echo "最新データ: {}"; jq -r '.data[-1].time' data/raw/{}'
          ls -ch data/processed/ -t -name'*.csv' | head -1 | xargs -I {} sh -c 'echo "最新予測: {}"; wc -l data/processed/{}'
```

パフォーマンス目標（重大修正後）:

- 総実行時間: 10分 以内（GitHub Actions無料枠考慮）
- LightGBM学習: < 30秒
- Keras学習: < 60秒
- API通信: < 10秒

エラー通知戦略:

- 失敗時のみ通知: GitHub Actionsの標準機能（成功時は通知なし）
- キャッシュ活用: `actions/cache@v3`で依存パッケージをキャッシュ

6. ローリング時系列交差検証（学習等未適合）

決定: 時系列データの特性を考慮し、ランダムシャッフルなしのローリング検証を採用

実装方法:

```
# 例: 2010-2022年のデータから最適な年組み合わせを探索
from itertools import combinations

years = range(2010, 2024)
best_combination = None
best_rmse = float('inf')

for combo in combinations(years, 3):
    # combo = (2010, 2011, 2012) のような組み合わせ
    train_data = load_data(combo)
    test_data = load_data(2013)  # 2013年をテスト

    model = train_model(train_data)
    rmse = evaluate_model(model, test_data)

    if rmse < best_rmse:
        best_rmse = rmse
        best_combination = combo

print(f"最適学習年: {best_combination}, RMSE: {best_rmse}")
```

理由:

- **時系列特性**: 電力需要は季節性（ヒンタ冬が決め）、トレンド（経済成長）を含む
- **未来予測**: 過去データのみを使って、将来の値を予測する必要
- **過学習防止**: 複雑組み合わせを比較し、テスト誤差が最小の組合

パスカルアクティブ:

- 固定テスト年: 2024年度（2025年）を固定し、評価年の公平性
- 上位10組み合わせ保存: 最適解だけでなく、上位候補も記録
- 結果の可視化: メモ保存（テキスト）、エラーバー付き棒グラフ

7. start-dashboard.ps1ワンクリック起動

決定: PowerShellスクリプトでサーバー起動とブラウザ起動を自動化

実装詳細:

```
# .vscode/tasks.json
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Start Server",
      "type": "shell",
      "command": "python -m http.server 8000",
      "isBackground": true,
      "problemMatcher": [],
      "presentation": {
        "reveal": "always",
        "panel": "dedicated"
      }
    }
  ]
}

# start-server.ps1 (クロスプラットフォーム)
Set-Location A1
& cmd /c "python -m http.server 8000"

# .vscode/settings.json
{
  "python.defaultInterpreterPath": "python",
  "python.terminal.activateEnvironment": false
}
```

理由:

- 開発効率: 毎日実行する作業が1クリック化され、ミス防止が容易に
- 再現性: 起動手順がドキュメントとして、常時参照可能な状態
- 学習容易: 初心者向けの手引き記述し、ドメイン知識の蓄積

パスカルアクティブ:

- 固定ポート: 8000番ポートを使用（競合時のみ変更）
- 上位10組み合わせ保存: 最適解だけでなく、上位候補も記録
- 結果の可視化: メモ保存（テキスト）、エラーバー付き棒グラフ

8. 16分割プロット生成（GitHub Pages対応）

決定: matplotlibでPNG形式で16分割グラフを生成

実装方法:

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figure=(16, 9))  # 16:9比率
ax.plot(hours, predictions, marker='o', linewidth=2, color='green')
ax.set_xlabel('時間 (JST)', fontsize=14)
ax.set_ylabel('電力需要 (kW)', fontsize=14)
ax.set_title('明日の電力需要予測', fontsize=16)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('A1/tomorrow/LightGBM/LightGBM_tomorrow.png', dpi=150)
plt.close()
```

理由:

- GitHub Pages対応: PNG形式でファイルをGitHub Pagesで直接表示可能
- 視認性: 16:9比率が各セルで一定幅で、全画面表示に最適
- ファイルサイズ: DPI 150で高品質なPNGのみ（ファイル容量小）

パスカルアクティブ:

- 日本語フォント: `rcParams['font.sans-serif'] = ['MS Gothic']`でWindows日本語対応
- 文字へのパス: スクリプトディレクトリ（`os.path.dirname(os.path.abspath(__file__))`）からの相対パス
- 凡例: モデル名と略語（ボ）を凡例に表示

技術スタック最終決定

フロントエンド

- HTML5/CSS3: クリックボイラー、スタンド後ろエフェクト
- JavaScript: 1D&4D、localStorage、非同期処理の管理
- Fetch API: HTTPサーバーの通信（`POST /log-trace`等）

バックエンド

- Python 3.10.11: 機械学習パイプライン
- http.server（標準ライブラリ）: HTTPサーバー（Port 8002）
- LightGBM 3.3.5: 高速・高精度ブースティング
- Keras/TensorFlow 2.13.0: 深層学習
- scikit-learn 3.0.0: RandomForest、データ前処理
- PyCaret 3.0.4: AutoML
- pandas 2.0.3: データ処理
- requests 2.31.0: Open-Meteo API通信

インフラ

- GitHub Pages: 静的ファイルホスティング
- GitHub Actions: CI/CD、毎日自動実行
- Git: バージョン管理、プッシュ自動（標準決定）

未解決事項

調査なし。すべての技術選定と設計判断が完了しました。

次のステップ

Phase 1（設計）へ進む。仕様書、UIワイヤフレームを生成します！

- data-model.md: エンティティ、フィールド、ワレンジ、ワンクリップ定義
- contracts/: API仕様（OpenAPI形式）
- quickstart.md: ワンクリップ起動手順

作成者: GitHub Copilot レビー状態: Completed 最終更新日: 2025-11-26